## DSN (Dağıtık Depolama Ağı) Mimari İncelemesi

Sistem, dosyaların yükünü (data transfer) ana sunucuya (backend) bindirmez. Direkt "Peer-to-Peer" (P2P) konseptinde (Uploader -> Storage Provider) doğrudan bir veri akışı sağlarken, merkezi sunucuyu sadece bir "Yönlendirici/Kimlik Doğrulayıcı (Metadata Koordinatörü)" olarak kullanır.

## 1. Temel Bileşenler ve Kullanılan Teknolojiler

Projede birbirini tamamlayan 4 ana bileşen bulunmaktadır:

1. **Frontend (Kullanıcı Arayüzü - Uploader)**
    a. **Görevi:** Kullanıcının (dosya yükleyen / X kişisi) platforma giriş yaptığı, dosyaları seçtiği, güvenlik operasyonlarını (şifreleme) ve ağ transferini gerçekleştiren web katmanı.
    b. **Teknolojiler:** Next.js 14, React 18, TypeScript, TailwindCSS, Shadcn/UI.
2. **Backend (Metadata & Koordinasyon Sunucusu)**
    a. **Görevi:** Sistemin beyni. Kimlik doğrulama, hangi dosyanın/chunk'ın hangi node'da (Agent) duracağını belirleme (manifest oluşturma), node sağlığını denetleme (heartbeat takibi) işlemlerini gerçekleştirir. Dosya içeriğini **kesinlikle** kendi diskine kaydetmez. Sadece SQL veritabanında "hangi parça kimde" (metadata) bilgisini tutar.
    b. **Teknolojiler:** Python 3.11, FastAPI (Asenkron API), PostgreSQL 15, Redis (Önbellek & Rate Limit), SQLAlchemy & Alembic (ORM ve Veritabanı Migrasyonu).
3. **Agent (Depolama Sağlayıcı - Storage Node)**
    a. **Görevi:** Gönüllülerin (Y kişisi) bilgisayarlarına kurdukları arka plan servisidir. Kendisine ayırdığı diski ağa açar ve dışarıdan gelen (Frontend'den gelen) şifreli chunk'ları depolar.
    b. **Teknolojiler:** Go (Golang) 1.21. İletişim için yerleşik hafif HTTP Server kullanır. Sistem tepsisi (tray) uygulamasını barındırır.
4. **Installer (Kurulum Sihirbazı)**
    a. **Görevi:** Storage Node olmak isteyen son kullanıcılara, karmaşık Go/CLI komutları yerine tıpkı standart bir program kurar gibi "İleri, İleri" mantığıyla Agent kurulumu yaptıran masaüstü uygulamasıdır.
    b. **Teknolojiler:** Tauri (Rust tabanlı) + React + Vite.

## 2. P2P (Doğrudan) Veri Aktarımı İşleyişi (Yükleme Akışı)

Sistem veriyi merkezi bir sunucuda tutmaktansa P2P (Uçtan Uca Doğrudan Bağlantı) yaklaşımı kullanır. Fakat bu aktarım BitTorrent veya WebRTC gibi karmaşık bir P2P protokolü ile değil, **Doğrudan HTTP (Direct HTTP Request)** mantığında çalışır.
**Bir Dosyanın Yüklenme Adımları:**

1. **İzin İsteme (Initial Request):** Tarayıcı (Frontend), Web API üzerinden Backend'e dosya boyutu, adı ve replikasyon (kaç güvenlik kopyası) isteğini iletir (`POST /files/upload/init`).
2. **Kapasite ve Manifest Tespiti:** Backend, veritabanından "`O an yeterli alan ve kota sahip aktif Agent'ların listesini`" çeker. Dosyayı (varsayılan) 16MB'lık hayali parçalara (chunk) böler gibi plan yapar. Frontend'e bir **Upload Manifest (Rota Planı)** döner. Bu planda şunlar vardır: "*Örn. Chunk 1'i Ali'nin ve Veli'nin (Agent) IP Adresindeki http://...:7777 URL'sine gönder! İşte bu da aranızdaki güvenlik anahtarları!*"
3. **Frontend Şifreleme (Web Crypto API):** Frontend dosyanın içeriğini RAM'a parça parça alır. Her bir 16MB'lık Chunk'ı tarayıcı içinde **AES-256-GCM** protokolüyle şifreler, üstüne **SHA-256** özetini çıkarır.
4. **P2P Doğrudan Yükleme:** Tarayıcı (Frontend), şifreli veriyi Backend'i atlayarak **doğrudan** ilgili Agent'ın (Storage Node) IP ve Portuna (`http://Agent-IP:7777/chunks/{chunk_id}`) HTTP PUT isteği ile yükler.
5. **Tamamlanma:** Tüm chunklar Agent'lara başarıyla kopyalandıktan sonra Frontend Backend'e "İşlem bitti" der (`/files/upload/complete`) ve Backend, Chunk'ların durumunu Veritabanında **Active** olarak günceller.

## 3. Mimari Kavramları ve Güvenlik Mekanizmaları

- **Zero-Trust Node Mimarisi (Sıfır Güven):** Depolama yapan Agent'a zerre kadar güvenilmez. Dosyalar, Agent'a ulaşmadan **önce** Frontend'de şifrelendiği için Agent sadece anlamsız **Byte** yığınları görür. (Uçtan Uca Şifreleme)
- **Chunking (Parçalama):** 5GB'lık devasa bir dosyayı indirmek/dağıtmak zor olacağı için veriler **16MB'lık standart** bloklara bölünür.
- **Replication (Çoğaltma - Replikasyon):** Sistem hiçbir dosyayı tek bir kişinin bilgisayarına teslim etmez. Veriler ağda (kullanıcının seçimine bağlı: Dengeli 2, Güvenli 3) olmak üzere birden fazla ajana dağıtılır. Çöken ajan olursa, sistem diğer kopyalardan otomatik olarak yeni ajanlara replikasyon (Re-replication) yapar.
- **Heartbeat & Sağlık (Health) Durumu:** Agent'lar (Go Uygulamaları) sürekli olarak Backend'e WebSocket/HTTP aracılığıyla 30 saniyede bir "*Ben buradayım, diskim çalışıyor*" mesajı atar. Bu da Backend'in metadata sisteminin canlı kalmasını sağlar.

## 4. Kim Ne ile İletişim Kuruyor?

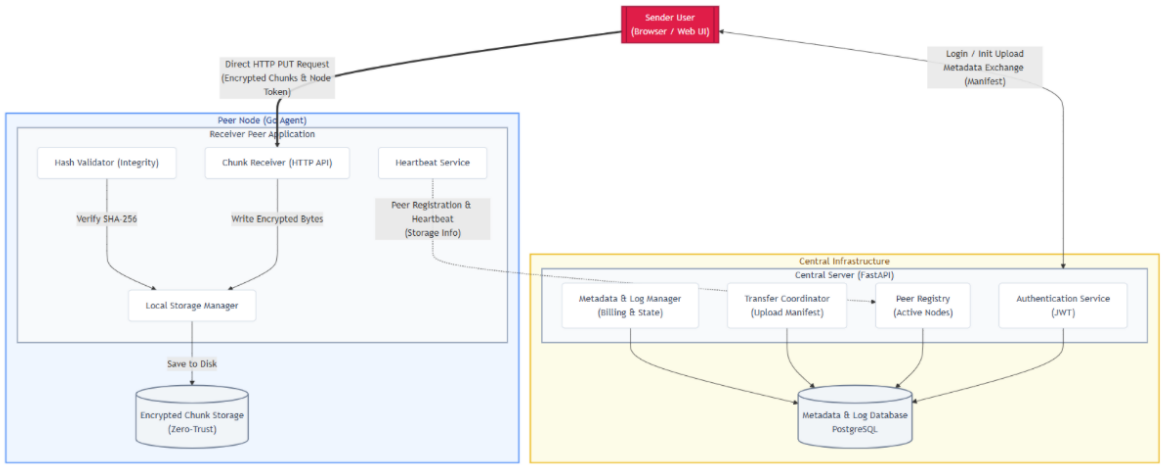
| Kaynak | Hedef | Protokol | Ne Taşınıyor? |
|---|---|---|---|
| **Tarayıcı (Uploader)** | Backend (FastAPI) | HTTP / REST | Yükleme rotası (Manifest), Metadata, JWT Kimlik Doğrulaması. |
| **Tarayıcı (Uploader)** | Agent Node (Go) | HTTP (Doğrudan) | **Şifrelenmiş Ham Veri (Chunks)** (Direct Upload/Download). |

| | | | |
|---|---|---|---|
| **Agent Node (Go)** | Backend (FastAPI) | HTTP / WebSocket | Sağlık durumu (Heartbeat) bildirimi, kullanılan alan verisi. |
| **Backend (FastAPI)** | Postgres & Redis | TCP / SQL | Veritabanı sorguları (metadata), Node istatistikleri. |

Kısacası, veri aktarımı özel bir P2P protokolü (BitTorrent vb.) ile değil; **Frontend'in, merkezi bir listeden (Backend'den) öğrendiği ağdaki dağınık cihazlara doğrudan HTTP isteği atmasıyla** dağıtık bir yapı (Distributed Storage) halinde sağlanmaktadır.

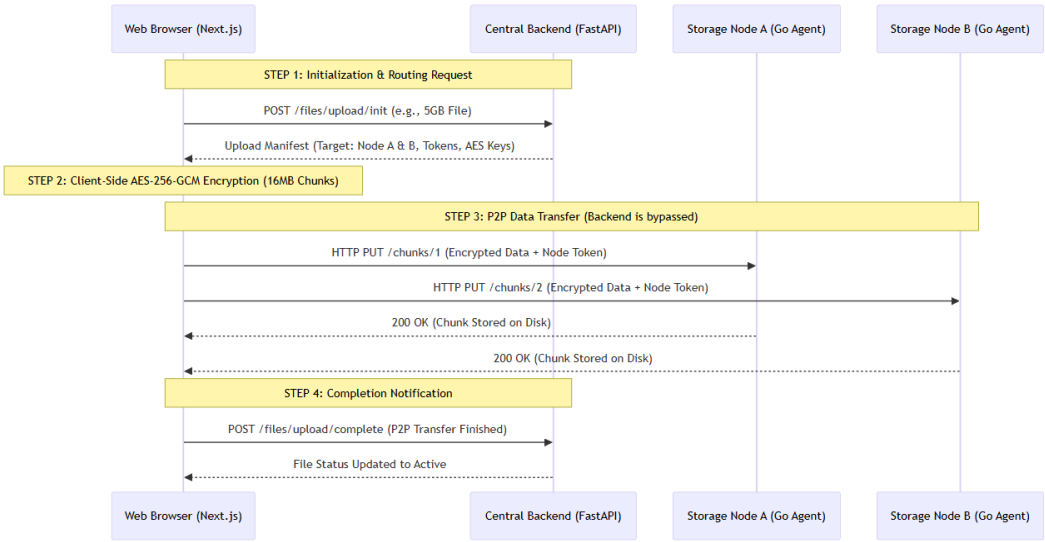
## A. Overall System Architecture Diagram

This diagram shows the main components of the Distributed Storage Network (DSN) and how they communicate.




## B. Uploader File Transfer Flow (Sequence Diagram)

This sequence diagram illustrates the step-by-step process of how a file is chunked, encrypted, and uploaded directly to the storage nodes, bypassing the central backend server for the actual data transfer.


Web Browser (Next.js)
Central Backend (FastAPI)
Storage Node A (Go Agent)
Storage Node B (Go Agent)
STEP 1: Initialization & Routing Request
POST /files/upload/init (e.g., 5GB File)
Upload Manifest (Target: Node A & B, Tokens, AES Keys)
STEP 2: Client-Side AES-256-GCM Encryption (16MB Chunks)
STEP 3: P2P Data Transfer (Backend is bypassed)
HTTP PUT /chunks/1 (Encrypted Data + Node Token)
HTTP PUT /chunks/2 (Encrypted Data + Node Token)
200 OK (Chunk Stored on Disk)
200 OK (Chunk Stored on Disk)
STEP 4: Completion Notification
POST /files/upload/complete (P2P Transfer Finished)
File Status Updated to Active
Web Browser (Next.js)
Central Backend (FastAPI)
Storage Node A (Go Agent)
Storage Node B (Go Agent)